

PS: note made from [introduction to computation and programming using python](#)

author: lencelg from Arcadia Bay

如果在字符串前加上r, 可以防止专业字符被转义

python使用一个符号表（常称为栈帧frame）。这个表跟踪记录函数中所有的名称定义（包括形参）和它们当前的绑定。

回文字符串判断

```
def isPalindrome(s):
    """假设s是字符串
    如果s是回文字符串则返回True，否则返回False。
    忽略标点符号、空格和大小写。"""

    def toChars(s):
        s = s.lower()
        letters = ''
        for c in s:
            if c in 'abcdefghijklmnopqrstuvwxyz':
                letters = letters + c
        return letters

    def isPal(s):
        if len(s) <= 1:
            return True
        else:
            return s[0] == s[-1] and isPal(s[1:-1])

    return isPal(toChars(s))
```

文件操作常用函数

`open(fn, 'w')` : `fn`是一个表示文件名的字符串。创建一个文件用来写入数据，返回文件句柄。

`open(fn, 'r')` : `fn`是一个表示文件名的字符串。打开一个已有文件读取数据，返回文件句柄。

`open(fn, 'a')` : `fn`是一个表示文件名的字符串。打开一个已有文件用来追加数据，返回文件句柄。

`fh.read()` : 返回一个字符串，其中包含与文件句柄`fh`相关的文件中的内容。

`fh.readline()` : 返回与文件句柄`fh`相关的文件中的下一行。

`fh.readlines()` : 返回一个列表，列表中的每个元素都是与文件句柄`fh`相关的文件中的一行。

`fh.write(s)` : 将字符串 `s` 写入与文件句柄`fh`相关的文件末尾。

`fh.writeLines(S)` : `S`是个字符串序列。将`S`中的每个元素作为一个单独的行写入与文件句柄`fh`相关的文件。

`fh.close()` : 关闭与文件句柄`fh`相关的文件。

list operations

`L.append(e)` : 将对象`e`追加到`L`的末尾。

`L.count(e)` : 返回`e`在`L`中出现的次数。

`L.insert(i, e)` : 将对象`e`插入`L`中索引值为`i`的位置。

`L.extend(L1)` : 将`L1`中的项目追加到`L`末尾。

`L.remove(e)` : 从`L`中删除第一个出现的`e`。

`L.index(e)` : 返回`e`第一次出现在`L`中时的索引值。如果`e`不在`L`中，则抛出一个异常（参见第7章）。

`L.pop(i)` : 删除并返回`L`中索引值为`i`的项目。如果`L`为空，则抛出一个异常。如果`i`被省略，则`i`的默认值为`-1`，删除并返回`L`中的最后一个元素。

`L.sort()` : 升序排列`L`中的元素。

`L.reverse()` : 翻转`L`中的元素顺序。

我们通常应该尽量避免修改一个正在进行遍历的列表

`assert Boolean expression`

或者：

`assert Boolean expression, argument`

执行`assert`语句时，先对布尔表达式求值。如果值为`True`，程序就愉快地继续向下执行；如果值为`False`，就抛出一个`AssertionError`异常。

$O(1)$ 表示常数运行时间。

$O(\log n)$ 表示对数运行时间。

$O(n)$ 表示线性运行时间。

$O(n \log n)$ 表示对数线性运行时间。

$O(n^k)$ 表示多项式运行时间，注意 k 是常数。

$O(c^n)$ 表示指数运行时间，这时常数 c 为底数，复杂度为 c 的 n 次方。

recursive binary search

```
def search(L, e):
    """假设L是列表，其中元素按升序排列。
    ascending order.
    如果e是L中的元素，则返回True，否则返回False"""
    def bSearch(L, e, low, high):
        #Decrements high - low
        if high == low:
            return L[low] == e
        mid = (low + high)//2
        if L[mid] == e:
            return True
        elif L[mid] > e:
            if low == mid: #nothing left to search, careful here
                return False
            else:
                return bSearch(L, e, low, mid - 1)
        else:
            return bSearch(L, e, mid + 1, high)

    if len(L) == 0:
        return False
    else:
        return bSearch(L, e, 0, len(L) - 1)
```

斐波那契数列 with memory, or you can just write them in a loop

```
def fastFib(n, memo = {}):
    """假设n是非负整数，memo只进行递归调用 返回第n个斐波那契数"""
    if n == 0 or n == 1:
        return 1
    try:
        return memo[n]
    except KeyError:
        result = fastFib(n-1, memo) + fastFib(n-2, memo)
        memo[n] = result
        return result
```

dynamic programming with 0/1 bag

```
def maxVal(toConsider, avail):
    """假设toConsider是一个物品列表，avail表示重量
    返回一个元组表示0/1背包问题的解，包括物品总价值和物品列表"""
    if toConsider == [] or avail == 0:
        result = (0, ())
    elif toConsider[0].getWeight() > avail:
        #探索右侧分支
        result = maxVal(toConsider[1:], avail)
    else:
        nextItem = toConsider[0]
        #探索左侧分支
        withVal, withToTake = maxVal(toConsider[1:],
            avail - nextItem.getWeight())
        withVal += nextItem.getValue()
        #探索右侧分支
        withoutVal, withoutToTake = maxVal(toConsider[1:], avail)
        #选择更好的分支
        if withVal > withoutVal:
            result = (withVal, withToTake + (nextItem,))
        else:
            result = (withoutVal, withoutToTake)
    return result

"""即使背包的容量很大，如果物品重量来自一个相当小的重量
集合，那么很多物品集合都会具有相同的总重量，这样就极大地缩短了程序运行时间。这称为伪多项式复杂度"""
```

本福德分布

令S是一个大的十进制数集合，那么每个非0数字出现的第一位的概率是多少？

对于一个十进制数的集合，如果第一位数字是d的概率符合

$$P(d) = \log_{10}\left(1 + \frac{1}{d}\right)$$